LangChain

- It is an open-source framework for building applications powered by Large Language Models (LLMs), enabling developers to create context-aware and scalable AI workflows.
- LangChain comes with a modular architecture which supports code reusability, flexibility, and easy switching of components like LLMs, data sources and other tools.

Key Points:

Purpose:

- Simplifies LLM application development by providing modular components (Chains, Agents, Prompts, Memory, Retrieval).

Integration:

- Connects LLMs with external data sources, APIs, tools, and vector databases to produce grounded, relevant answers.

Flexibility:

- Supports multiple LLM providers (OpenAI, Anthropic, HuggingFace, etc.) and vector stores (FAISS, Pinecone, Chroma).

Need for LangChain

Challenges of Using LLMs Directly

Statelessness

Each LLM API call is independent; it does not remember past interactions. Developers must manually maintain and send context and chat history with every request.

Lack of Grounding

LLMs can hallucinate and don't access up-to-date or real-time data, resulting in less accurate responses.

Inability to Act

LLMs generate text only (input text → output text). They cannot execute code, trigger APIs, or perform actions to complete tasks beyond text generation.

Complex Workflow Management

Manually coordinating multi-step workflows, conditional logic, error handling, and integration across diverse systems is challenging and error-prone.

LangChain Addressing these Challenges

Standardized Interfaces

LangChain provides modular, reusable components-chains, agents, retrievers, memory modules-that manage context, retrieval, and external tool use seamlessly.

Runtime Orchestration

Automates workflow logic including branching, looping, retries, and error handling to make complex AI pipelines manageable and robust.

Retrieval Integration for Grounding

Combines LLMs with vector stores and external data sources to ground responses in real knowledge, reducing hallucinations.

Dynamic Action Execution

Enables agents to call external tools, APIs, or execute code dynamically during interactions, overcoming the text-only limitation.

LangChain Ecosystem

A modular, extensible framework designed to simplify building advanced large language model (LLM) applications by integrating models, tools, data connectors, and orchestration layers.

Core Libraries in the Ecosystem

Library	Purpose
langchain-core	**LangChain Core** is the foundational library of LangChain that provides the **essential abstractions, interfaces, and runtime components** for building LLM-powered applications, such as prompts, models, retrievers, and chains, without additional integrations or extras.
langchain-community	Community-maintained integrations including vector stores, document loaders, models, and APIs to keep core package lightweight.
Integration-packages	Important integrations have been split into lightweight packages that are co-maintained by the LangChain team and the integration developers.(e.g. langchain-openai, langchain-anthropic, etc.)

Why Use the LangChain Ecosystem?

Modular & Extensible Design

Enables building customized workflows by mixing core components with community integrations while keeping the base lightweight.

Supports Complex AI workflow

Facilitates orchestration of multi-step pipelines and multi-agent systems for advanced use cases through langgraph and agents.

Production-Ready Tooling

Integrates with tools like **LangSmith** for workflow monitoring, evaluation, and debugging, plus **LangServe** for scalable API deployment.

Broad Ecosystem Integration

Connects easily with vector stores, databases, external APIs, and various models to leverage existing infrastructure effectively.

LangChain-Chains

- Chains are ordered sequences of modular steps that connect LLM calls, data processing, and external tools to create reusable AI workflows
- Chains enable building complex workflows by linking simple components step-by-step.

LangChain Chain Types

1.Sequential Chains

- Linear workflows where the output of one step feeds directly into the next.
- Ideal for simple pipelines with a clear step-by-step process.

2. Conditional (Branching) Chains

- Dynamically selects the next step based on specific conditions or outputs from prior steps.
- Supports decision-making and alternate execution paths in workflows.

3. Parallel Chains

- Executes multiple chains concurrently on the same input or different inputs.
- Useful for tasks that can be run independently and combined later.

Sequential Chain code snippet

```
from langchain.chains import SimpleSequentialChain

chain = SimpleSequentialChain(chains=[embed_step, retrieve_step, llm_step])
result = chain.run("User Query")
```

Here we're importing *SimpleSequentialChain* and define a chain with three steps: embedding the user query, retrieving relevant information, and generating a final answer with an LLM.
When we run the chain, it processes the query sequentially, and the *result* holds the AI-generated response.

Prompt Templates & Custom Prompts in LangChain

01 Prompt Templates

- Help **translate user input and parameters** into formatted instructions for language models.
- Guide's the model's responses by providing **context** and structure to generate better outputs.
- It accepts a **dictionary** as input, where each key corresponds to a variable in the template.

02 Types of Prompt Templates

- **StringPromptTemplate** Formats plain string prompts; good for simple inputs.
- **ChatPromptTemplate** Formats a **list of messages**; consists of multiple prompt templates for chat-based interactions.

03 Custom Prompts

- User-defined templates or instructions to control LLM responses or actions.
- Created using the *PromptTemplate* class in LangChain.
- Enable building **complex workflows** where prompts vary based on earlier steps or external data.

LangChain Memory

- ❖ Memory stores and recalls past conversation data, letting LLM apps maintain context across multiple interactions.
- ❖ It lets LangChain retain and use past context which is essential for chatbots, virtual assistants, and any multi-turn AI app.

Importance of Memory

- Enables multi-turn conversations without forgetting previous messages.
- Makes interactions more natural and human-like.
- Can store both short-term (recent turns) and long-term (persistent) data.

Main Types of Memory:

- **ConverstationBufferMemory:** Keeps full conversation history.
- **ConverstationBufferWindowMemory: :** Stores only the last N exchanges.
- **ConverstationSummaryMemory:** Summarizes past interactions.
- **VectorStoreRetrieverMemory:** Retrieves context from a vector database

```
from langchain.memory import
ConversationBufferMemory

# Initialize conversation memory buffer
memory = ConversationBufferMemory()

# Save a sample conversation turn (input
and output)
memory.save_context({"input": "Hi"}, {"output":
"Hello!"})

# Load current conversation history from
memory
print(memory.load_memory_variables({}))
# output is {'history': 'Human: Hi\nAI:
Hello!'}
```

LangChain Document Loaders

- Components that import and preprocess documents from various external data sources.
- Convert data (PDFs, websites, emails, databases, etc.) into standardized document objects.
- Handle tasks like file reading, format parsing, OCR (for scanned docs), and chunking.

Why Document Loaders Matter

- Prepare raw data for semantic search, retrieval, and LLM consumption.
- Enable LangChain to work with **diverse data types and sources**.
- Key first step in building retrieval-augmented workflows and knowledge applications.

Common Document Loader Types in LangChain:

- **PDFLoader:** Loads and extracts text from PDF files.
- **UnstructuredURLLoader**: Fetches and parses web pages.
- **CSVLoader:** Reads and processes CSV datasets.
- **TextLoader:** Loads plain text files.
- **EmailLoader:** Parses email formats.

Code snippet

```
from langchain.document_loaders
import PDFLoader

# Load PDF and extract text
loader = PDFLoader(“data.pdf")
documents = loader.load()

# Display content of first document chunk
print(documents[0].page_content)
```

LangChain Retrievers

- Retrievers are components that fetch relevant documents from a data source based on a query, without using traditional keyword search - usually leveraging embeddings and similarity search.
- They are the **bridge between a LLM and your data,** ensuring responses are accurate, relevant, and context-aware.

Why they're used?

- To provide the **right context** to LLMs for grounded answers and to ensure efficiency & scalability for large knowledge bases.
- To Work with any backend: vector stores, databases, API indexes. .

Common Retriever Types:

- **VectorStoreRetriever:** Retrieves items by nearest-neighbor search on embeddings.
- **TimeWeightedVectorStoreRetriever: :** Prioritizes recent documents.
- **MultiQueryRetriever:** Reformulates queries into multiple variations to improve recall.

LangChain Vector Stores

- Vector stores are databases optimized for **storing and retrieving vector embeddings** (numerical representations of text, images, etc.) using similarity search.
- They are the **memory banks for embeddings,** allowing LangChain to retrieve the most relevant information based on meaning, not just words.
- Some of the popular Vector Store Integrations are **FAISS, Pinecone, Chroma**, etc.

Importance of Vector Stores

- Enables semantic search instead of keyword matching.
- Powers the retrieval step in RAG (Retrieval-Augmented Generation) workflows.
- Scalable for large document collections.

Vector Store Working

- The documents are chunked and converted into numerical vectors for storage ;these vectors are then stored in a special, searchable database.
- Then the question/query is also converted into a vector using the same model.
- The store finds the closest document vectors to answer user specific question/query.

LangChain Expression Language(LCEL)

- A **declarative language** for creating LangChain workflows called "chains" by describing *what* should happen rather than *how*.

- Enables clean, readable pipeline composition using the pipe (|) operator for chaining components.

Key Features:

- Supports **optimized execution** with parallelism, asynchronous calls, and streaming outputs.
- Chains implement the Runnable interface, ensuring flexibility and seamless integration within LangChain.
- Fully compatible with LangChain's observability tools (e.g., LangSmith) and deployment platforms (e.g., LangServe).

Example Syntax:

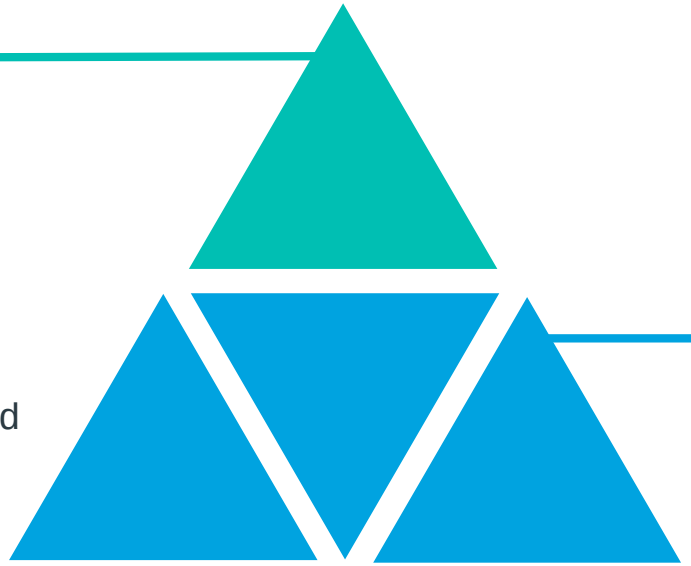
```
lcel_chain = prompt_template | llm | output_parser  # defining the LCEL chain(prompt → LLM → output parser)
result = lcel_chain.invoke("Your input query")       # run the chain on the query and get the result
```

Retrieval-Augmented Generation(RAG)

It's a technique that combines external knowledge retrieval with LLM text generation to produce accurate, up-to-date, and context-aware responses..

RAG working in LangChain

- **Load Data**; Bring documents from files, web, or databases into LangChain.
- **Create Embeddings:** Turn document chunks into vector form for semantic search.
- **Store & Search:** Save vectors in a vector store and find the closest matches to your query.
- **Generate Answer**: Give the retrieved text to the LLM to create a final, context-aware response.



Benefits

- Uses current & domain-specific knowledge without retraining the model.
- Reduces hallucination by grounding answers in retrieved facts.
- Flexible – works with structured, unstructured, or API data sources.

Limitations of LangChain

1

Learning Curve

Requires understanding of LLM workflows, agents, memory, retrieval, vector stores—can be complex for beginners.

2

Dependency on External Models and APIs

Performance depends heavily on external LLM providers (e.g., OpenAI), which may add latency, cost, and rate limits

3

Scalability Challenges

Some components may need custom optimization for very large-scale or real-time production environments.

4

Tooling Maturity

Some advanced features (multi-agent orchestration, LCEL ecosystem) are evolving and might lack extensive documentation or community support

5

Error Handling & Hallucinations

LLM outputs can be unpredictable, it requires careful prompt design, evaluation, and monitoring to reduce errors